Formal Review Request Template

Requester: Nishanth Manoj
Reviewer: Gloria Obeng
Date: 11/02/26
Approx. review duration: 65 min

Choose **one primary review method**:

Code / Software Review

Code Quality Review
Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.

White-Box / Edge-Case Testing
Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.

Black-Box / Functional Testing
Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.

Performance / Runtime Testing
Systematically evaluate whether the software meets relevant runtime or performance expectations.

Usability Review

- **Task-Based Usability Testing**
  Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.
- **Heuristic / Guidelines Evaluation**
  Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.
- **Prototype Usability Testing**
  Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

Design Review

- **Architecture / Conceptual Design Review**
  Evaluate whether architecture and major design decisions align with project requirements and constraints.
- **Preliminary Design / UML Review**
  Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.
- **Security Design Review**
  Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

Other

- **Other review method:** _____
  _____

Explain what the reviewer will evaluate and why this method is appropriate for your work:

Since our project has a lot of different files and features, I thought it would be better to focus on a smaller part of the code. I chose the semester creation form because it has a few different fields that could be checked. Just making sure the code is organized well and there aren't any mistakes or things that could be improved.

Review Request Details

1) What specifically should the reviewer examine or interact with?

The main file I would like to have reviewed is app/views/semesters/new.html.erb. This includes the semester and year dropdowns, CSV upload fields, and the submit button. I would like the reviewer to go through these sections and check if everything is set up correctly and if there are any possible issues.

2) What question do you want this review to help answer?

Examples:

- Does this implementation correctly handle boundary and error cases?
- Can a new user complete the checkout task without assistance?
- Does this architecture satisfy our project's stated requirements?

I mainly want to find out if there are any issues with how the semester creation form was written. Whether some parts of the code are confusing, repeated, or could be simplified. Also if the form fields are properly set up and follow good coding practices.

3) What should the reviewer focus on?

Some of the main things to check would be the dropdowns, CSV upload fields, and the overall organization of the form. I would also like them to look for any repeated code, possible mistakes, or areas that could be cleaned up to make the code easier to maintain.

4) What should the reviewer *not* evaluate during this review?

Examples:

- Do not evaluate visual polish or mobile responsiveness (for a usability test evaluating navigation and task completion)
- Do not evaluate the database layer or unfinished API integration (for a code review focusing on the authentication module)

Since this review is focused on one section of the project, there is no need to go through the other features such as team management or student authorization. The database and backend logic can also be left out since the main focus is on the semester creation form and its code.

5) What should the reviewer use to judge the work?

Examples include expected software behavior, project requirements, coding best practices, usability heuristics, or design constraints.

I think general coding practices and the expected functionality of the form would be appropriate. The reviewer can check whether the code is readable, the fields are labeled correctly, and if there are any unnecessary sections or possible issues that could affect the form later.

Reviewer Section

1) Briefly describe what you actually did during the review.

I reviewed the *app/views/semesters/new.html.erb* file as a code quality review. I checked the semester and year dropdowns, the CSV file upload fields and their labels, the form structure, error handling, and the overall organization of the ERB/HTML code. I also checked for unnecessary or repeated code within the view.

2) What portions of the artifact did you evaluate?

I evaluated the semester selection dropdown, year selection dropdown, CSV upload fields for the student roster, client, and Git files, the optional CSV upload heading, form submission button, validation-error display, and the overall form layout and ERB structure.

---

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.

https://github.com/uofm-capstone/fall2026_tagem_up

Findings

Strengths:

- The semester dropdown is clearly defined with `Fall`, `Spring`, and `Summer`.
- The year dropdown provides a range from three years before to three years after the current year.
- The current semester and year are preserved when validation errors occur.
- The three CSV fields have descriptive labels: Student Roster CSV, Client CSV, and Git CSV.
- The CSV fields use `accept: 'text/csv'`, which communicates the expected file type to the browser.

Issues / Defects:

Description of Issue	Severity (Low/Med/High)	Evidence / Example	Suggestions
The labels for the semester	Low	`<label for="semester">Semester</label>` and `<label`	Consider using `f.label`

and year use plain HTML rather than Rails form labels.		`for="year">Year</label>`	consistently, as is already done for the CSV fields.
The semester options are defined inside the view	Low	`semester_array = %w[Fall Spring Summer]`	Consider defining these options in a more reusable location if they are needed elsewhere.
The three CSV upload blocks are structurally repetitive	Low	Each column repeats the same `form-outline,` label, and `file_field` pattern.	If the project expects more CSV fields later, consider using an array and iterating over the fields to reduce duplication.

Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.

The feedback pointed out a few things in the semester creation form that I could improve. I will look into using Rails form labels for the semester and year fields to keep the code more consistent. Also moving the semester options somewhere more reusable if needed. For the CSV upload fields, I think the repeated code is fine for now since there are only three fields, but it could be simplified later if more are added. Overall, the feedback helped point out some areas that could be cleaned up and made easier to maintain.
